

System programming 12th project

Introduction:

In this project the 5 compiler phases should be implemented using any programming language of your choice. The projects' groups should consist of **at most 3 members from the same class**.

Project Guidelines:

The following guidelines must be fulfilled, which are:

- 1- Each team will select one of the available projects through a google form (*will be submitted by your teaching assistant*)
- 2- Your project must include the following phases:
 - a. Context free grammar
 - b. Lexical analysis
 - c. Syntax analysis
 - d. Semantic analysis
 - e. Optimized intermediate code generation (*3 address code*)
- 3- Members of each group should be assigned to the previous phases as follows:
 - a. Context free grammar generation + lexical analyzer (1 member)
 - b. Syntax analyzer + semantic analyzer (1 member)
 - c. Optimized intermediate code generation using TAC (1 member)
- 4- Your input file to be compiled must include ***one of the statements*** (*as in topic 5 – available statements*) as well as ***variable declarations*** and ***expressions***.
- 5- The available projects are as follows:

Syntax analyzer	Available statements
Left most derivation	If / if-else
Right most derivation	While loop
LL parser (top-down)	Do-while loop
LR parser (bottom-up)	For loop
	Switch – case - default
	Break – continue in loops
	Functions
	Array manipulation

Project Requirements:

The following requirements must be fulfilled in the project, which are:

- 1- Generate the appropriate **Context Free Grammar (CFG)** for the selected statements including expressions and variable declarations.
- 2- **Lexical analyzer** must break down each input line into tokens after removing whitespaces and comments.
- 3- **Syntax analyzer** using any method of the selected methods (*as in topic 5 – Syntax analyzer*) must generate syntax tree in JSON format (*abstract syntax tree – AST*).
- 4- **Semantic analyzer** must do the following:
 - a. Check if all used variables are declared.
 - b. Check for no repeated variables.
 - c. Check if all variables are compatible in expression.

d. Generate Symbol Table.

- 5- **Optimized intermediate code generation** must generate the code in three address code (TAC) and then optimize it using any optimization technique.

Project Evaluation:

Points will be awarded to members with a higher understanding of the implementation. The authenticity of the implementation will also be awarded higher grades than implementations downloaded from the internet. The more phases a group finishes the more points they will accumulate. Finally, fully downloaded codes are not advised, if two different groups submitted similar code in any phase, it will be handled as a cheating case and both groups will be awarded zero.

Project Deliverable:

The following documentation must be submitted for evaluation by the end of week 11. No late submissions will be accepted. The required documentation is:

1. A report illustrating the different phases included in your compiler. The report must include all the design steps in details.
2. PowerPoint presentation.
3. Source code for the project in any language of your choice.

Bonus

Bonus points will be awarded for the **best 4 projects** among all projects as follows:

- 1st **2 points**
- 2nd **1.5 points**
- 3rd **1 point**
- 4th **0.5 point**